great expectations

THE BEGINNER'S GUIDE TO DATA QUALITY TESTS

What kinds of testing to deploy, and why

Table of contents

So you're implementing a data quality and observability process...

Congratulations on taking the first step toward having confidence in your data!

In this guide, we walk through the most critical dimensions of your data's quality and recommend a set of tests for each.

Why trust our recommendations? Because we've been where you are. Great Expectations was founded by data practitioners who needed a better way to do better data quality tests.

And over the years, we've built that better way: our open source data quality framework, GX Core, is the most popular data quality tool in the world.

In the process of making GX Core what it is today we've talked to a lot of data practitioners, listened to their problems, and learned from their challenges. The GX platform—GX Core plus our SaaS offering, GX Cloud—is a distillation of what we've learned from our 12,000+ community members.

And even better, many members of the GX team have been in your shoes. As former data practitioners, building GX Core and GX Cloud is an opportunity for them to help create the solution that they themselves needed.

About anomaly detection

Anomaly detection is an extremely popular kind of data quality. The key to effectively using anomaly detection is to deploy it strategically and in support of your other tests. Unfortunately, it's often not deployed this way, and it ends up a hindrance to your overall data quality and observability efforts.

Anomaly detection's main appeal is that it's extremely standardized and automated, so it's easy to apply tests to a lot of data quickly. But because it's so standardized, anomaly detection tests are low-information.

Things anomaly detection tests can tell you	Things anomaly detection tests can't tell you
The anomaly's characteristics When the anomaly was detected What data the anomaly was detected in	The significance of the detected anomaly The urgency of the anomaly The importance of the data that has the anomaly The stakeholders of the data that has the anomaly

Their ease of application at scale makes it easy to overuse anomaly detection tests. And by applying anomaly detection too widely, you end up inundated by an enormous number of minimally informative alerts.

Triaging those anomaly detection test results and tracking down the context you need to take effective action (or not) can quickly start taking up way too much of your time.

To avoid this, we recommend applying anomaly detection tests judiciously. Focus on using them to:

- Implement basic monitoring for select, critical sources you haven't been able to create high-information tests for yet.
- Establish baseline behavior for sources you're unfamiliar with when you don't have access to a knowledgeable stakeholder.

Ultimately, your goal should be to replace your anomaly detection tests with high-context, information-rich specific tests that provide you with everything you need to take appropriate action.

The basics

Here are the dimensions we recommend for a starter set of high-information data quality tests—with the goal of each dimension's checks in simple, jargon-free language for your nontechnical stakeholders.

#1: Missingness

Goal: Check that data isn't unexpectedly missing—and doesn't unexpectedly exist.

We recommend starting with null checks.

Even if your pipeline carries out null checks when data is initially loaded into it, data can accidentally be changed or deleted once it's in.

Plus, the point of having a data quality and observability process is to document your knowledge about your data in one central place, and that includes your expectations around nulls.

#2: Schema

Goal: Check that the columns you expect to be there are present and in the order you expect them to be.

Schema changes are one of the most common problems that we hear about.

Data that originates from outside your team might undergo an unannounced schema change.

And even within your own systems, with complex pipelines, it's all too easy for slightly misconfigured queries to drop or add columns, for a small column name or type change to send ripple effects downstream, or any number of other issues.

#3: Volume

Goal: Check that the amount of data you're receiving is within the bounds you expect: not too few rows and not too many.

The informativeness of this test depends heavily on the specific data you're looking at. At one extreme, you know exactly the number of rows that you're expecting; at the other, you're checking that there are rows.

Data volume checks are particularly critical for operational systems where an increase in data volume can require bringing additional computational resources online. If additional resources come online automatically, you'll want to be notified so you can monitor the cost increase. If you need to manually activate more resources, you'll want to be notified so you can do that.

#4: Ranges

Goal: Check that numbers and dates are within the ranges you expect them to be.

Like volume tests, the degree of specificity you can employ for numeric and datetime range checks depends heavily on the specific data you're checking.

Even if you know there are specific requirements, you might not have enough information at the start to do anything other than look for reasonable orders of magnitude. This is completely fine! You'll learn more about the data as you iterate and will be able to refine your tests.

GX Cloud starter set

If you're using GX Cloud for your data quality and observability testing, you'll implement your tests as Expectations: expressive, human-readable assertions about what you expect your data to look like.

Expectations we recommend to cover "the basics" as we've described here are:

- Expect column values to be null
- Expect column values to not be null
- Expect table columns to match set
- Expect table row count to be between
- Expect column max to be between
- Expect column min to be between

Taking it to the next level

Effective data quality and observability processes are iterative—because your data and its uses aren't static.

This is why we recommend starting out with basic testing and gradually scaling up. Short iterative cycles that are as frequent as you can make them minimize the chances that your tests are out of date before you've finished implementing them.

This is also why Expectations in GX Cloud and GX Core are so collaboration-friendly: because their power comes from combining simple tests to reflect complex requirements, you don't get code spaghetti even when they're continually modified by multiple people.

Schema tests

Schemas that change without warning are a major irritant to just about anyone who uses data, and data quality practitioners are no exception.

If you've been able to configure your pipelines to be resilient to certain kinds of schema changes, you can deprioritize or omit tests for those kinds of changes. Type checking is a useful safeguard against configuration changes or renames, but this is easy to omit if your data isn't consistently typed.

Option 1: Test column names, order, and type.

This is a strict but straightforward set of tests. Choose this combination of checks if your downstream uses for the dataset depend on column name, order, and type.

Option 2: Test column names and type.

This is a less restrictive set of checks. It's particularly useful if your downstream uses for the data rely entirely on column headers and type, ignoring order.

Option 3: Test individual columns by name and type.

This set of checks is useful when you are only interested in a small subset of the columns in a dataset. This is doubly true if the dataset's schema is also volatile and checking the complete schema would prompt constant alerts.

Option 4: Test the column count.

This test is very broad, making it most informative for datasets that don't have reliable and/or consistent column headers.

GX CLOUD EXPECTATIONS FOR SCHEMAS

- Expect table column to match ordered list
- Expect table columns to match set
- Expect column to exist
- Expect column values to be of type
- Expect column values to be in type list
- Expect table column count to be between

Volume tests

Testing your data to make sure it's present in the quantities you expect is an essential part of a data quality process.

Option 1: Test the row count against fixed values.

When you expect your data volumes to remain consistent over the medium-to-long term, test them against a fixed value.

With expressive tests, you can easily specify a fixed range for your data volume to be compared to if needed. This is the difference between testing for 800 rows and testing for 700-900 rows. If specifying a fixed range is difficult, it's a sign that the tests you're using lack expressivity, and you will likely need to explore other options.

Option 2: Test the row count against dynamic values.

When you expect your data volumes to remain relatively consistent in the short term but to potentially vary medium-to-long term, test them against a dynamic value (or a dynamic range of values) that's calculated using past results.

GX CLOUD EXPECTATIONS FOR VOLUME

- Expect table row count to be between

Validity tests

We call these checks *validity* and not *correctness*. Why? Because it's important to be extremely thoughtful about what *correctness* (or *accuracy*, if you prefer) means.

Correctness means you're comparing your data to a truth. If your test doesn't specify or refer to that source of truth, it's not really testing correctness; it's testing whether the data is plausible (aka valid). You might discover in your data quality process that some invalid values are correct or that some valid values are never correct.

FINITE SET TESTS

For non-numeric, non-datetime data where you expect the value to be (or not to be) one of a comparatively small set of values, you can use set-based checks to establish your data's validity.

Option 1: Check that the value belongs to a set.

This is useful in a wide variety of scenarios. When information is decomposed into multiple fields for convenience in technical handling, be sure to carry out this check across multiple columns as a unit.

Option 2: Check that a value does not belong to a set.

This is less common but is still helpful in some situations, such as checking that an address' country is not on a list of embargoed countries.

Option 3: Check that the most common value (and only the most common value) in a column is one of a set.

This approach can be informative if you have a situation where you have a fixed set of values where you expect some to always be uncommon: for example, an 'other' or 'none of these' value.

GX CLOUD EXPECTATIONS FOR SET-BASED VALIDITY

- Expect column values to be in set
- Expect column pair values to be in set
- Expect column values to not be in set
- Expect column most common value to be in set

NUMERIC AND DATETIME

For numeric- and datetime-typed data, set-based checks are typically impractical, if not impossible. Instead, statistical tests are useful for checking the validity of meaning.

Option 1: Check the column range.

This is the simplest check and is extremely common.

Option 2: Test the column distribution.

Some common ways of characterizing values' distribution are mean, median, quantiles, standard deviation, and z-score.

Option 3: Test the column sum.

This check is either essential or nonsensical—rarely anything in between.

GX CLOUD EXPECTATIONS FOR NUMERIC AND DATETIME VALIDITY

- Expect column values to be between
- Expect column max to be between
- Expect column min to be between
- Expect column mean to be between
- Expect column median to be between
- Expect column quantile values to be between
- Expect column standard deviation to be between
- Expect column z-scores to be less than
- Expect column sum to be between

SHAPE TESTS

Shape tests are most useful when the number of possible valid values is too large for finite set checks to be usable: for example, any field that's originated in full or in part by free text entry. If you are using finite set checks, you may find shape checks to be superfluous.

The exception is if your testing approach doesn't allow you to immediately see values that failed to pass a test.* In that case, you can often use shape checks to identify potentially valid but currently invalid values to investigate.

Option 1: Test the value length.

Most fields can benefit from some form of this, whether the range of allowed lengths is narrow or wide.

Option 2: Test that the value matches a pattern.

Defining effective pattern-matching can be challenging, so expect to need to iterate on these checks!

Option 3: Test that the value does not match a pattern.

As with other non-match tests, this is less common but can be useful for ensuring that certain types of data are not present: for example, looking for social security number patterns to ensure that SSN data has been successfully redacted.

* If you're a GX Cloud user, you don't have to worry about this; GX Cloud shows values that failed an Expectation directly alongside the test result by default.

GX CLOUD EXPECTATIONS FOR SHAPE VALIDITY

- Expect column value length to be between
- Expect column value length to equal
- Expect column values to match like pattern
- Expect column values to match like pattern list
- Expect column values to match regex
- Expect column values to match regex list
- Expect column values to not match like pattern
- Expect column values to not match like pattern list
- Expect column values to not match regex
- Expect column values to not match regex list

Uniqueness tests

Uniqueness checks are ubiquitous. They generally don't make sense paired with finite set validity checks, but otherwise can deliver valuable information about a wide variety of data.

Option 1: Test for value uniqueness.

This is yet another occasion where knowing your data's business context is important.

In addition to checking for the positive—verifying that data that *should* be unique *is* unique—you can also use these tests to look for data that's too unique. Setting a relatively low uniqueness requirement is a good way to monitor for suspicious similarities in data you expect to be extremely variable.

GX CLOUD EXPECTATIONS FOR UNIQUENESS

- Expect column values to be unique
- Expect column unique value count to be between
- Expect column proportion of unique values to be between

Referential integrity testing

Referential integrity across columns and across tables verifies whether data that is related to other data has the right relationship. It's an essential part of a robust data quality process.

Option 1: Test cross-column references

A common use for this type of check is when a single piece of real-world information is represented as several data fields. For example, a `day` field that contains a `31` shouldn't have a corresponding `month` field that indicates `February`, `April`, `June`, `September`, or `November`.

Option 2: Test cross-table references

Checking cross-table references can be particularly informative for shaping your data quality priorities. Do data quality issues occur consistently across tables? If they don't, why not? And if they do, is there a common source for the problematic data?

GX CLOUD EXPECTATIONS FOR REFERENTIAL INTEGRITY

- Expect column pair values to be equal
- Expect multicolumn sum to equal
- Expect compound columns to be unique
- Expect table row count to equal other table

Conclusion

Building out comprehensive data quality testing for your data doesn't have to be daunting. Especially since anomaly detection tests aren't the cure-all they're hyped up to be, leaving you with an avalanche of notifications and still uninformed about what your data needs to look like.

Only expressive and explicit data quality tests can enable you to build a process that sets you up for immediate and effective action when you have data quality issues.

The tests we shared in "The basics" guide you to a starting set of checks that can catch common fundamental errors in your data's missingness, schema, volume, and value ranges.

With the basics in place, you can use your own experience with the data and/or insight from the data's stakeholders to take your tests to the next level, with more nuanced tests for the kinds of problems that are most frequent or have the most impact.

By creating a suite of expressive and explicit data quality tests, you'll have high-context and information-rich alerts that provide everything needed to take immediate and appropriate action whenever a data quality issue happens.

great expectations

GX Cloud's Expectations are the most expressive way to implement data quality tests.

Plus, they're human-readable in plain language, so it's easy for data stakeholders to understand them no matter their level of technical ability. And when all your stakeholders can understand the data quality work directly, collaboration is easier, your tests get better and more comprehensive, and your data quality process is stronger.

You can set up a GX Cloud account and be testing your data in just minutes. Sign up to try it for yourself, or request a demo.

START TRUSTING YOUR DATA

greatexpectations.io